Data wrangling with dplyr

POLI 210 - Seminar 8

Brad LeVeck
October 21, 2026

Questions to answer with data

- How do observed offers and MAOs differ between elites and undergraduates?
- How much could missing MAOs change a sample mean?
- Which retained House races were close? How can we attach NOMINATE scores?

Today: use a small set of functions repeatedly, with checks along the way.

Reading: PSD Chapter 6. Follow along in the live-demo QMD.

First, identify the observation

Table	One row represents
`ug`	One elite or undergraduate participant
`elections`	One retained district-election
`members`	One member record in the 114th Congress

The rows are not interchangeable. Joining or reshaping can change what one row means.

Before calculating: **What is the unit? What identifies it? What is measured?**

Our ultimatum-game data

`ug`: 102 elites and 132 undergraduates. Same study as B2, with more variables.

- `mao`: minimum acceptable offer; `offer`: proposed offer. Both use 0–100.
- Each respondent's two choices are recorded in the same row.
- We retain missing responses. Zero is a valid response.

Elite status was not randomly assigned. Group comparisons are descriptive.

LeVeck et al. (2014), Harvard Dataverse, doi:10.7910/DVN/27894. The original incentives involved lotteries; our interpretation uses the simplified 100-point game.

Our congressional-election data

The selected course file has 9,337 district-elections, 1946–2014.

Column	Meaning
`dv`, `dvp`	Democratic two-party vote percentage, current/prior election
`inc3`	1 Democratic incumbent, −1 Republican incumbent, 0 no incumbent
`year`, `stcd`	Election year and state-district code

Not a complete election census: 328 retained races in 2014, not 435. Do not call an unweighted mean of `dv` the national popular vote.

Gary Jacobson (2015), replication archive, doi:10.7910/DVN/29559. The local CSV is a selected extract; definitions checked against the archive codebook.

Open the project and load the data

```
library(dplyr)
library(readr)
ug <- read_csv("data/ug.csv")
elections <- read_csv("data/elections.csv")
```

Open `poli210-seminar8.Rproj` first. `data/ug.csv` means “inside this project’s data folder.”

Install packages once; load them each session. The demo’s setup loads all five tables.

Data frames and tibbles

```
glimpse(ug)     # Columns, types, and example values
names(ug)      # Column names
dim(ug)        # Rows and columns
head(ug)       # First few rows
as_tibble(ug)  # Tibble representation
```

A tibble is a data frame with convenient printing and consistent subsetting.

Only seeing ten rows in the Console does **not** mean the other rows disappeared.

Filter observations

```
filter(elections, year == 2014, inc3 == 0)
filter(ug, population == "Elites", mao >= 50)
filter(ug, is.na(mao))
```

- Commas combine conditions with **AND**; `|` means OR.
- `==` compares; `%in% c(2010, 2014)` checks membership.
- `filter()` keeps `TRUE`, dropping `FALSE` and `NA`.

Use `is.na(mao)`, not `mao == NA`.

Select columns and rename them

```
select(ug, participant_id, population, mao, offer)
select(ug, -experience)
rename(elections, dem_vote = dv)
```

`filter()` chooses **rows**. `select()` chooses **columns**.

`rename(new_name = old_name)` changes a label without removing other columns.

These calls return a result. They do not overwrite the input unless you assign it.

Read a pipeline as “then”

```
open_2014 <- elections %>%
  filter(year == 2014, inc3 == 0) %>%
  select(year, stcd, dv, dvp) %>%
  arrange(desc(dv))
```

Start with elections, **then** keep open-seat races in 2014, **then** choose columns, **then** sort them.

`x %>% f(y)` usually means `f(x, y)`. The book uses `%>%`; `|>` also works here.

Sort versus retain extreme rows

```
elections %>% filter(year == 2014) %>% arrange(dv)
elections %>% filter(year == 2014) %>% slice_max(dv, n = 3)
elections %>% filter(year == 2014) %>% slice_min(dv, n = 3)
```

`arrange()` reorders. `desc()` reverses the order.

`slice_max()` and `slice_min()` select rows. Ties are retained by default: `n = 3` can return more than three rows.

Mutate: numerical transformations

```
elections_derived <- elections %>%
  mutate(
    dem_share = dv / 100,
    dem_swing = dv - dvp,
    distance_from_50 = abs(dv - 50)
  )
```

Same rows, new columns. `dv` remains available.

`dem_share` is a proportion; `dem_swing` is a percentage-point change. Distance from 50% is half the two-party victory margin.

Mutate: flags and categories

```
ug_flags <- ug %>% mutate(
  mao_missing = is.na(mao),
  offers_half_or_more = offer >= 50,
  sample_label = if_else(elite == 1, "Elite", "Undergraduate")
)
```

`if_else(condition, yes, no)` makes a choice for **each row**.

For several conditions, use `case_when()`; see the demo's incumbent-party labels. Labels must follow the codebook.

Mutate: bounds for missing responses

In `ug`, **four elite MAOs are missing; no elite offers are missing**.

```
ug_bounds <- ug %>% mutate(
  mao_lower = if_else(is.na(mao), 0, mao),
  mao_upper = if_else(is.na(mao), 100, mao),
  offer_lower = if_else(is.na(offer), 0, offer),
  offer_upper = if_else(is.na(offer), 100, offer)
)
```

Preserve observed responses. Replace only missing values in **new columns**. `coalesce(mao, 0)` is shorthand for the lower-bound transformation.

What do those bounds tell us?

Elite sample	Observed-case mean	All missing = 0	All missing = 100
MAO	30.6	29.4	33.4
Offer	43.0	43.0	43.0

The observed MAO mean uses 98 people. **Both bounds use all 102.**

Width: $100 \times 4/102 = 3.92$ points. Offer bounds coincide because nothing is missing.

These bound the finite-sample mean under the 0–100 restriction. They are **not** confidence intervals or guesses about anyone's actual missing response.

Distinct values are not distinct people

```
ug %>% distinct(population)
elections %>% distinct(year, stcd)
ug %>% distinct(mao)
```

The first gives sample labels. The second gives district-election keys.

The third gives distinct **MAO values**, including missingness. It does not give a deduplicated sample of participants.

Two people choosing 50 are still two observations.

Try a short pipeline

Among retained 2014 races, find the five closest to 50% Democratic vote.

Show `year`, `stcd`, `dv`, and `distance_from_50`.

Start from `elections_derived`. Use `filter()`, `slice_min()`, and `select()`.

Before running: predict what one row of the result will represent.

Summarize and count missing values

```
ug %>% summarize(
  n_rows = n(),
  n_mao = sum(!is.na(mao)),
  mean_mao = mean(mao, na.rm = TRUE)
)
```

`summarize()` collapses rows into summaries. `summarise()` is the same function.

`n()` counts every row. `na.rm = TRUE` uses observed MAOs only for that mean. Always be clear about the denominator.

Group by population, then summarize

```
ug %>%
  group_by(population) %>%
  summarize(n_rows = n(),
            n_mao = sum(!is.na(mao)),
            mean_mao = mean(mao, na.rm = TRUE),
            .groups = "drop")
```

Before: one row per person. After: one row per population.

Now try summarizing `mao_lower` and `mao_upper` by population. Why do the groups have different widths for their bounds?

Grouped mutate keeps the people

```
ug %>%
  group_by(population) %>%
  mutate(group_mean = mean(mao, na.rm = TRUE),
         mao_centered = mao - group_mean) %>%
  ungroup()
```

`summarize()`: group-level rows.

`mutate()`: group-based calculations attached to the original rows.

The mean of `mao_centered` among observed values is zero **within each group**.

Remember to ungroup

```
elections %>%
  group_by(year, inc3) %>%
  summarize(n_races = n()) %>%
  ungroup() %>%
  slice_max(n_races, n = 1)
```

Without `ungroup()`, this summary remains grouped by year, so the maximum would be selected **within each year**.

Alternatively, use `summarize(..., .groups = "drop")`.

B2's empirical PMF, written as a pipeline

```
ug %>%
  filter(!is.na(mao)) %>%
  count(population, mao, name = "frequency") %>%
  group_by(population) %>%
  mutate(probability = frequency / sum(frequency)) %>%
  ungroup()
```

`count()` counts rows at each population-MAO combination.

The probabilities must sum to 1 **within each population**. To get its CDF: sort MAOs, then cumulatively sum probabilities within the group.

Break and checkpoint

What is the difference between:

- `filter()` and `select()`?
- `mutate()` and `summarize()`?
- `mean(mao, na.rm = TRUE)` and `mean(mao_lower)`?

After the break: text, reshaping, and combining tables.

Character strings

```
library(stringr)
members %>%
  filter(str_detect(bioname, "PELOSI|SÁNCHEZ")) %>%
  mutate(surname = str_extract(bioname, "^[^,]+")) %>%
  select(bioname, surname)
```

`str_detect()` tests a pattern. `str_extract()` returns the first match.

Here `|` means OR; `^[^,]+` extracts text before the first comma. Use names for display and inspection, **not as an assumed unique join key**.

Pivot from person to person-choice rows

```
ug_long <- ug %>%
  select(participant_id, population, mao, offer) %>%
  pivot_longer(cols = c(mao, offer),
               names_to = "choice", values_to = "amount")
```

`mao` and `offer` become values in `choice`. The numbers become `amount`.

One row per person becomes one row per person-choice: **468 rows, still 234 people.**

Same information, different shape

Person	MAO	Offer
eu-0001	37	40
eu-0002	40	50

Person	Choice	Amount
eu-0001	mao	37
eu-0001	offer	40
eu-0002	mao	40
eu-0002	offer	50

Choose a shape for the question. A row-count change is not a sample-size increase.

Widen a summary table

```
ug_long %>%
  group_by(population, choice) %>%
  summarize(mean_amount = mean(amount, na.rm = TRUE),
            .groups = "drop") %>%
  pivot_wider(id_cols = population,
              names_from = choice, values_from = mean_amount)
```

Each population-choice combination must identify one value before widening.

Do not replace a missing mean with zero just to fill a table cell.

Appending observations

```
all_ug <- bind_rows(
  ug %>% select(participant_id, population, mao, offer),
  turk %>% select(participant_id, population, mao, offer)
)
all_ug %>% count(population)
```

`bind_rows()` stacks observations and matches **column names**.

The original MTurk file repeats the elite block. The provided `turk` table already excludes it. Equal responses are not duplicates to delete.

Joining adds information by a key

```
elections %>% left_join(states, by = "state")
```

Elections `state`	Lookup `state`	`state_name`	`state_abbrev`
1	1	Alabama	AL
4	4	Arkansas	AR
5	5	California	CA

This code is alphabetical position among the 50 states. It is **not** Voteview's ICPSR code or a FIPS code. Matching column names do not guarantee matching meanings.

Match the election to the right Congress

```
elections_2014 <- elections %>%
  filter(year == 2014) %>%
  mutate(district = stcd %% 100, congress = 114) %>%
  left_join(states, by = "state", relationship = "many-to-one")
```

2014 election → 114th Congress (2015–2017).

`members` is already restricted to that House term. This describes subsequent representation, not the pre-election incumbent.

Use Congress, state abbreviation, and district together.

Check for multiple matches

```
members %>%
  count(congress, state_abbrev, district_code) %>%
  filter(n > 1)
```

Six districts have two member records during the 114th Congress.

For example, Illinois 18 has Aaron Schock and Darin LaHood. A district-election does not by itself identify which member row you intended.

Do not use the first row as an undocumented tie-breaker.

A left join can increase the row count

```
raw_join <- elections_2014 %>%
  left_join(members,
    by = c("congress", "state_abbrev", "district" = "district_code"))
```

In our data: **328 election rows become 333 rows**.

A left join retains all left observations, but repeats them for multiple matches. Later averages would give those districts extra weight.

Check rows and keys even when R gives no warning.

A transparent rule for ambiguous districts

```
members_unique <- members %>%
  group_by(congress, state_abbrev, district_code) %>%
  filter(n() == 1) %>%
  ungroup()
```

For today's example, attach member information only when the district has a single member record in that Congress.

Leave ambiguous election rows in place, with missing member information. This is a conservative classroom rule, not a general method for identifying winners.

Join with an explicit expectation

```
joined <- elections_2014 %>%
  left_join(members_unique,
    by = c("congress", "state_abbrev", "district" = "district_code"),
    relationship = "many-to-one")
```

The relationship check requires at most one right-side match per left row.

It does not guarantee a match or establish that the key is the right one. It requires dplyr 1.1.0 or newer.

Audit the result

```
anti_join(elections_2014, members_unique,
  by = c("congress", "state_abbrev", "district" = "district_code"))
```

- Still **328** election rows; **323** have a unique member match.
- Five ambiguous districts remain: HI-1, IL-18, KY-1, MS-1, PA-2.
- Separately check matched members whose NOMINATE score is missing.

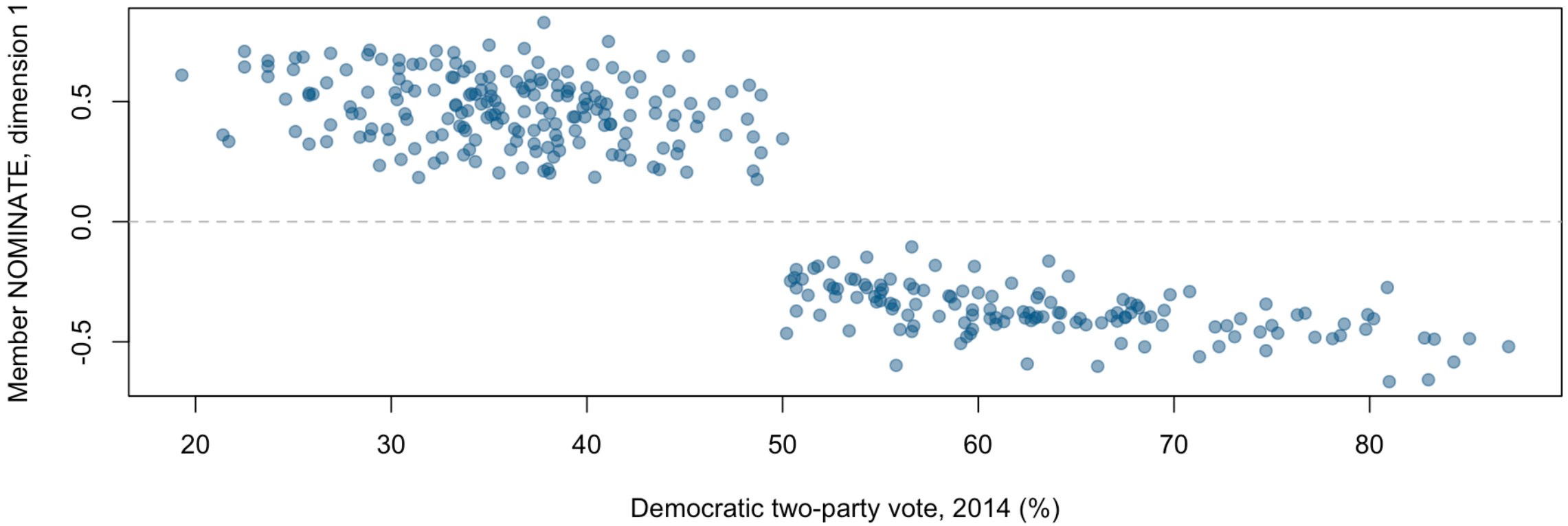
Missing match ≠ missing measurement ≠ zero ideology.

Other joins answer different questions

Function	Rows retained
`left_join(x, y)`	All left observations, adding matching right fields
`inner_join(x, y)`	Matches only, adding right fields
`right_join(x, y)`	All right observations
`full_join(x, y)`	Observations from either side
`semi_join(x, y)`	Left rows with a match; left fields only
`anti_join(x, y)`	Left rows without a match; left fields only

Choose based on the intended observations, not which command removes the NAs.

The merged data, described



Selected elections with a unique subsequent member record. NOMINATE summarizes roll-call positions, not voters' ideology; this is descriptive, not causal. Jacobson + Voteview, frozen October 20, 2026.

A reproducible result

```
write_csv(joined, "elections-2014-with-nominate.csv")
```

Keep raw inputs intact and save the code that produced the result.

Before using a table: check its unit, missingness, keys, and row count.

The demo is a reference: all core examples run locally, without downloads.

Reference: the rest of the string toolkit

Function	Result
`str_length()` / `str_count()`	Character length / number of pattern matches
`str_extract()` / `str_extract_all()`	First match / all matches
`str_remove_all()`	Text with matching pieces removed
`str_detect()`	TRUE/FALSE for each string
`str_split()` / `str_split_fixed()`	Pieces in a list / a fixed-column matrix

The demo uses actual member names. Pattern matching needs care with accents and naming conventions. Cleaning labels does not validate a merge.

Optional: apply and sapply

```
amounts <- as.matrix(select(ug, mao, offer))
apply(amounts, MARGIN = 2, FUN = mean, na.rm = TRUE)

sapply(c(20, 30, 40, 50), function(o) {
  mean(ug$mao <= o, na.rm = TRUE)
})
```

`apply()`: repeat across matrix columns (`2`) or rows (`1`). `sapply()`: repeat over elements; here, offers in a simplified UG calculation.

Do not mix text columns into a numeric matrix. PSD §6.7 is useful for reading other people's code.

Reading and references

- Speegle and Clair, PSD Chapter 6.
- dplyr, grouped operations, joins, tidyr pivoting, stringr.
- Jacobson (2015), House-election replication data.
- LeVeck et al. (2014), elite bargaining replication data.
- Lewis et al. (2026), Voteview.

Detailed definitions, preparation choices, and source links accompany the demo.